[MARCADOR: LOGOTIPO ITSQMET]

INSTITUTO SUPERIOR TECNOLÓGICO

TALLER INTEGRADOR NÚCLEO 1

Informe del Taller 5 Caso Integrador

Gestión de usuarios productos y pedidos

Estudiante: [MARCADOR: NOMBRE DEL ESTUDIANTE]
Carrera: Desarrollo de Software
Docente: [MARCADOR: NOMBRE DEL DOCENTE]
Fecha: 9 de octubre de 2026

Introducción

Este informe documenta la implementación del Taller 5 del Núcleo 1 de ITSQMET. El proyecto integra Spring Boot, Spring MVC, Spring Data JPA, MySQL, Thymeleaf, una API REST y Spring Security para resolver un caso de gestión de usuarios, productos, categorías y pedidos.

La solución fue construida de forma incremental sobre el proyecto existente. Las pruebas automatizadas se ejecutaron con H2 en memoria y la instancia demo se inició correctamente en el puerto 8080. La conexión final a MySQL queda parametrizada por variables de entorno y requiere validar las credenciales del equipo de entrega.

Problemática

La guía oficial plantea una empresa que actualmente utiliza hojas de cálculo y registros manuales. Este mecanismo favorece la duplicidad de información, dificulta la trazabilidad y no ofrece un control adecuado de accesos. El sistema implementado centraliza la información y aplica permisos diferenciados para las operaciones administrativas.

Objetivos

Objetivo general

Implementar una aplicación web segura y persistente para administrar usuarios, productos, categorías y pedidos mediante una arquitectura MVC basada en Spring Boot.

Objetivos específicos

- Modelar las entidades y relaciones con JPA.
- Implementar operaciones CRUD para productos y categorías.
- Registrar usuarios con contraseñas cifradas mediante BCrypt.
- Exponer una API REST con autorización por método HTTP y rol.
- Validar el funcionamiento mediante pruebas automatizadas y pruebas HTTP.

Alcance del proyecto

- Registro e inicio de sesión de usuarios.
- Roles USER y ADMIN.
- Gestión de categorías y productos.
- Búsqueda y cálculo de unidades y valor del inventario.
- Registro y consulta de pedidos.
- Endpoints REST para productos, categorías y pedidos.
- Vistas Thymeleaf con Bootstrap.

Tecnologías utilizadas

Tecnología	Uso
Java 21	Lenguaje de desarrollo
Spring Boot 4.1.1	Configuración y ejecución
Spring MVC y Thymeleaf	Interfaz web
Spring Data JPA e Hibernate	Persistencia

Tecnología	Uso
MySQL	Base de datos de entrega
H2	Pruebas y perfil demo
Spring Security	Autenticación y autorización
Maven Wrapper	Construcción y pruebas

Arquitectura MVC

La aplicación se organiza en capas. Los controladores reciben las solicitudes, los servicios concentran las reglas de negocio, los repositorios acceden a la base de datos y las entidades representan el modelo persistente. Las vistas Thymeleaf conforman la capa de presentación y los controladores REST exponen recursos JSON.

Capa	Implementación
Model	Usuario, Categoria, Producto, Pedido, Role y EstadoPedido
Repository	Repositorios JpaRepository
Service	Reglas de registro, CRUD, búsquedas y cálculos
Controller	MVC web, autenticación y REST
View	Plantillas login, registro, productos y pedidos

Estructura del proyecto

La estructura relevante se encuentra dentro de src/main/java/com/example/tallernucleo1:

- config/SecurityConfig.java
- model/
- repository/
- service/
- controller/
- src/main/resources/templates/
- src/test/java/
- src/test/resources/application-test.properties

Modelo de datos y relaciones JPA

Usuario se relaciona con Pedido mediante una relación uno a muchos. Categoria se relaciona con Producto mediante una relación uno a muchos. Pedido y Producto se relacionan mediante una relación muchos a muchos con la tabla intermedia pedido_producto.

Entidad	Campos principales	Relación
Usuario	id, username, password, nombreCompleto, role	1:N con Pedido
Categoria	id, nombre	1:N con Producto
Producto	id, nombre, descripcion, precio, stock	N:1 con Categoria y N:M con Pedido
Pedido	id, fecha, direccionEntrega, estado	N:1 con Usuario y N:M con Producto

Entidades repositorios servicios y controladores

Los repositorios extienden JpaRepository y usan consultas derivadas para búsquedas. ProductoService resuelve la categoría, actualiza productos y calcula el valor total del inventario. PedidoService valida los productos seleccionados y asocia el pedido al usuario autenticado. Los controladores separan la navegación web de los endpoints REST.

Interfaz web

La interfaz incluye inicio de sesión, registro, listado y búsqueda de productos, formulario de alta y edición, eliminación protegida por sesión, creación de pedidos y consulta de pedidos. Bootstrap se utiliza para la estructura visual y la adaptación básica a diferentes tamaños de pantalla.

[MARCADOR DE EVIDENCIA: insertar captura de la página principal y del formulario de producto]

API REST

Método	Ruta	Acceso
GET	/api/productos y /api/productos/{id}	Público
POST	/api/productos	ADMIN
PUT	/api/productos/{id}	ADMIN
DELETE	/api/productos/{id}	ADMIN
GET	/api/categorias	Público
POST	/api/categorias	ADMIN
GET/POST	/api/pedidos	Usuario autenticado

Spring Security

La configuración mantiene CSRF activo para la aplicación web y lo ignora únicamente para /api/**, donde las pruebas utilizan HTTP Basic. Las rutas GET públicas de productos y categorías no requieren autenticación. Las operaciones POST, PUT y DELETE administrativas requieren ROLE_ADMIN. Las contraseñas se codifican con BCrypt y el usuario ADMIN inicial se crea desde variables de entorno.

Pruebas realizadas y resultados

Funcionalidad	Procedimiento	Esperado	Obtenido	Estado
Compilación	.\mvnw.cmd test	Sin errores	Exitosa	OK
Contexto Spring	Prueba de contexto	Cargar contexto	H2 cargado	OK
GET productos	GET /api/productos	200 público	200	OK
POST sin credenciales	POST /api/productos	401	401	OK
POST ADMIN	POST /api/categorias Basic	201	201	OK
POST USER	POST /api/categorias Basic	403	403	OK
Inicio demo	spring-boot:run demo	Iniciar	Tomcat 8080	OK
MySQL	Configuración predeterminada	Conectar	Pendiente de credenciales	PENDIENTE

Las pruebas automatizadas corresponden a ApiSecurityTests y TallerNucleo1ApplicationTests. Las pruebas de API adicionales se ejecutaron contra la instancia demo local mediante solicitudes HTTP.

Evidencias visuales

Se verificaron visualmente, mediante navegador, las siguientes pantallas: inicio de sesión con confirmación de registro, página principal de productos vacía y página de productos con el producto de prueba, stock 10 y valor de inventario 459.00.

- [CAPTURA PENDIENTE: estructura del proyecto en IDE]
- [CAPTURA PENDIENTE: tablas y relaciones en MySQL]
- [CAPTURA PENDIENTE: ejecución de pruebas Maven]
- [CAPTURA PENDIENTE: respuesta REST en Postman con 200, 201, 401, 403 y 404]

No se incluyen imágenes simuladas. Las capturas pendientes deben obtenerse en el equipo de entrega con MySQL activo y Postman/IDE abiertos.

Conclusiones

La implementación cubre la arquitectura MVC, persistencia JPA, CRUD de productos y categorías, gestión de pedidos, autenticación, autorización por roles, BCrypt, API REST y pruebas automatizadas. La instancia demo funciona y permite demostrar los flujos principales sin depender de una instalación externa.

La validación final contra MySQL y la incorporación de la carátula institucional permanecen pendientes porque no se dispuso de las credenciales de base de datos ni del archivo CARATULA.docx referenciado.

Recomendaciones

- Configurar DB_USERNAME y DB_PASSWORD sin escribirlos en archivos versionados.
- Tomar las capturas pendientes con la aplicación conectada a MySQL.
- Reemplazar los marcadores de carátula antes de entregar el informe.
- Cambiar ADMIN_PASSWORD para la demostración final.

Referencias documentales

Guía de la Unidad de Integración Curricular Integración de la Información, Taller 5 Caso Práctico.

Transcripciones N1_Clase_01_05Oct, N1_Clase_02_06Oct, N1_Clase_03_07Oct y N1_Clase_04_09Oct.